

DESARROLLO DE SOFTWARE BACKEND II

Procedimientos Almacenados

ALUMNA:

ERENDIRA D. BECERRA ACOSTA

DOCENTE:

GONZALEZ DIAZ ANSELMO ALEXIS

25 / SEP / 2026 /.

PROCEDIMIENTOS ALMACENADOS

❖ Actividad 1

Script:

```
USE e_commerce;
GO

ALTER PROCEDURE [dbo].[sp_GetAllActiveProducts]
   @method VARCHAR(100),
   @isActive BIT = NULL,
   @SKU VARCHAR(50) = NULL
AS
BEGIN
   SET NOCOUNT ON;

   IF @method = 'Products'
   BEGIN

      IF @isActive IS NULL
      BEGIN
         SELECT
            '0' AS code,
            'No se ha especificado el parámetro isActive' AS message;
         RETURN;
      END

      SELECT
         ProductId,
         SKU,
         Name,
         Description,
         Price,
         StockQuantity,
         '1' AS code,
         'Success' AS message
      FROM Products
```

```
    WHERE IsActive = @isActive
    ORDER BY Name ASC;

    RETURN;
END

IF @method = 'GetProductBySKU'
BEGIN

    IF @SKU IS NULL OR LTRIM(RTRIM(@SKU)) = ''
    BEGIN
        SELECT
            '0' AS code,
            'No se ha especificado el parámetro SKU' AS message;
        RETURN;
    END

    IF NOT EXISTS (
        SELECT 1
        FROM Products
        WHERE SKU = @SKU
    )
    BEGIN
        SELECT
            '0' AS code,
            'El producto no existe' AS message;
        RETURN;
    END

    SELECT
        @isActive = IsActive
    FROM Products
    WHERE SKU = @SKU;

    IF @isActive = 0
    BEGIN
        SELECT
            '0' AS code,
```

```
            'El producto ya no se encuentra activo' AS message;
        RETURN;
    END

    SELECT
        ProductId,
        SKU,
        Name,
        Description,
        Price,
        StockQuantity,
        '1' AS code,
        'Success' AS message
    FROM Products
    WHERE SKU = @SKU
      AND IsActive = 1
    ORDER BY Name ASC;

    RETURN;
  END

  SELECT
    '0' AS code,
    'El método especificado no es válido' AS message;
END;
GO
```

Prueba:

Se realizó una búsqueda de producto utilizando un SKU existente.

```
EXEC sp_GetAllActiveProducts @method = 'GetProductBySKU', @SKU = 'LAP-001';
```

Resultado

La prueba se ejecutó correctamente y mostró la información del producto

❖ Actividad 2

Script:

```
USE e_commerce;
GO

CREATE OR ALTER PROCEDURE [dbo].[sp_ValidateApiKey]
   @KeyValue UNIQUEIDENTIFIER
AS
BEGIN
   SET NOCOUNT ON;

   IF @KeyValue IS NULL
   BEGIN
      SELECT
         '0' AS code,
         'No se ha especificado el valor de la API Key' AS message;
      RETURN;
   END

   IF NOT EXISTS (
      SELECT 1
      FROM ApiKeys
      WHERE KeyValue = @KeyValue
        AND IsActive = 1
   )
   BEGIN
      SELECT
         '0' AS code,
         'La API Key no es válida o se encuentra inactiva' AS message;
      RETURN;
   END

   SELECT
      KeyName,
      '1' AS code,
```

```
    'Success' AS message
  FROM ApiKeys
  WHERE KeyValue = @KeyValue
    AND IsActive = 1;
END;
GO
```

Prueba:

Se realizó una prueba utilizando una API Key existente y activa en la base de datos.

```
EXEC sp_ValidateApiKey @KeyValue = 'API-KEY-DE-PRUEBA';
```

Resultado

La prueba se ejecutó correctamente y la API Key fue validada.

❖ Actividad 3

Script:

```
USE e_commerce;
GO

CREATE OR ALTER PROCEDURE [dbo].[sp_CreateOrder]
  @UserId INT,
  @ProductId INT,
  @Quantity INT
AS
BEGIN
  SET NOCOUNT ON;

  IF @Quantity <= 0
  BEGIN
    SELECT
```

```
        '0' AS code,
        'La cantidad debe ser mayor a cero' AS message;
    RETURN;
END

IF NOT EXISTS (
    SELECT 1
    FROM Users
    WHERE UserId = @UserId
)
BEGIN
    SELECT
        '0' AS code,
        'El usuario no existe' AS message;
    RETURN;
END

IF NOT EXISTS (
    SELECT 1
    FROM Products
    WHERE ProductId = @ProductId
      AND IsActive = 1
)
BEGIN
    SELECT
        '0' AS code,
        'El producto no existe o se encuentra inactivo' AS message;
    RETURN;
END

DECLARE @Stock INT;
DECLARE @Price DECIMAL(18,2);
DECLARE @TotalAmount DECIMAL(18,2);
DECLARE @OrderId INT;

SELECT
    @Stock = StockQuantity,
    @Price = Price
```

```
FROM Products
WHERE ProductId = @ProductId
  AND IsActive = 1;

IF @Stock < @Quantity
BEGIN
    SELECT
        '0' AS code,
        'No hay suficiente stock para realizar la orden' AS message;
    RETURN;
END

SET @TotalAmount = @Price * @Quantity;

BEGIN TRY

    BEGIN TRANSACTION;

    INSERT INTO Orders
    (
        UserId,
        TotalAmount,
        Status
    )
    VALUES
    (
        @UserId,
        @TotalAmount,
        'Pending'
    );

    SET @OrderId = SCOPE_IDENTITY();

    INSERT INTO OrderDetails
    (
        OrderId,
        ProductId,
        Quantity,
```

```
        UnitPrice
    )
    VALUES
    (
        @OrderId,
        @ProductId,
        @Quantity,
        @Price
    );

    UPDATE Products
    SET StockQuantity = StockQuantity - @Quantity
    WHERE ProductId = @ProductId;

    COMMIT TRANSACTION;

    SELECT
        @OrderId AS OrderId,
        @TotalAmount AS TotalAmount,
        '1' AS code,
        'Orden creada correctamente' AS message;

  END TRY
  BEGIN CATCH

    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    SELECT
        '0' AS code,
        ERROR_MESSAGE() AS message;

  END CATCH
END;
GO
```

Prueba:

Se realizó una prueba creando una orden para el usuario 1, utilizando el producto 1 y una cantidad de 2.

```
EXEC sp_CreateOrder @UserId = 1, @ProductId = 1, @Quantity = 2;
```

Resultado

La prueba se ejecutó correctamente.

❖ Actividad 4

Script:

```
USE e_commerce;
GO

CREATE OR ALTER PROCEDURE [dbo].[sp_GetUserForLogin]
   @Username NVARCHAR(50)
AS
BEGIN
   SET NOCOUNT ON;

   IF @Username IS NULL OR LTRIM(RTRIM(@Username)) = ''
   BEGIN
      SELECT
         '0' AS code,
         'No se ha especificado el nombre de usuario' AS message;
      RETURN;
   END

   IF NOT EXISTS (
      SELECT 1
      FROM Users
      WHERE Username = @Username
```

```
  )
  BEGIN
    SELECT
      '0' AS code,
      'El usuario no existe' AS message;
    RETURN;
  END

  SELECT
    UserId,
    PasswordHash,
    Role,
    '1' AS code,
    'Success' AS message
  FROM Users
  WHERE Username = @Username;
END;
GO
```

Prueba:

Se realizó una búsqueda de usuario utilizando un nombre de usuario existente.

```
EXEC sp_GetUserForLogin @Username = 'admin_user';
```

Resultado

La prueba se ejecutó correctamente y se obtuvo la información del usuario.